

The data engineer, in depth

Building and moving the data every other role runs on

M Usman

A close look at the profession the rest of the data stack stands on.

Where this is heading

The role is quiet, central, and short of people.

M Usman · From enterprise data to production AI.

The least visible role, and the one everything depends on

- Big-data specialists, the data-engineering family, rank among the **fastest-growing roles to 2030** (WEF, 2025), and the measured global AI wage premium has reached **56%** (PwC, 2025).
- The work is foundational, not optional:
 - Every dashboard an analyst builds and every model a scientist trains reads from tables an engineer designed, moved, and tested
 - **Python and SQL are the floor**; cloud has moved from a differentiator to an expectation; streaming and the lakehouse are the fastest-rising requirements
 - When the pipeline is right, nobody notices; when it breaks, every number downstream is wrong at once

Hire the engineer for the failure you will never see, not the dashboard you will.

The slow step

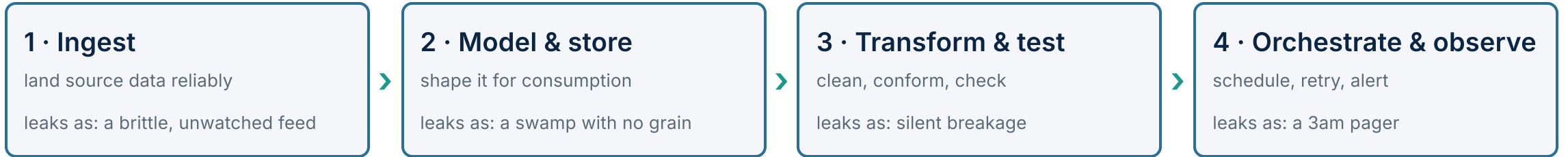
Data that cannot be trusted, breaking where nobody is looking

The engineer's value is not moving data; it is moving data that arrives complete, on time, with a known shape, every time. The failure is silent: a renamed column, a half-loaded file, a duplicated key, and the corruption flows downstream into a report or a model before anyone notices.

The engineer is measured by what does not go wrong. The four stages that follow are where that reliability is built in, or lost.

M Usman · From enterprise data to production AI.

The work, end to end



The four stages are one job: source data turned into trusted, fresh, business-ready tables under a system that watches itself. The next four slides take each in turn.

M Usman · From enterprise data to production AI.

1 • Ingest: land source data reliably

Source systems, APIs, files, and event streams pulled into the platform without losing a record or trusting a schema that can change without warning.

Practical steps

- Decide batch or streaming per source from the freshness the decision needs, not from habit
- Capture change rather than reload in full where the source allows it; make loads idempotent so a re-run is safe
- Enforce a **schema contract** at the boundary: quarantine a malformed load rather than passing it on
- Record load metadata, row counts, watermarks, and the source version, so a gap is visible

The pitfall: a brittle, unwatched feed. An upstream change or a partial extract lands unchecked, and the error surfaces three layers downstream, hours later.

Illustrative tools: Airbyte, Fivetran, Kafka, Debezium (change capture) · cloud-native loaders · schema registries.

M Usman · From enterprise data to production AI.

2 · Model & store: shape it for consumption

Raw data organised into a layered, queryable store with a deliberate grain, so consumers read clean, modelled tables instead of raw extracts.

Practical steps

- Land raw (bronze), clean and conform (silver), model for consumption (gold): the medallion pattern
- Choose the store for the workload: a **warehouse** for SQL analytics, a **lakehouse** where scale and open formats matter
- Fix the **grain** of every table and document it; partition and cluster for how the data is actually queried
- Store in columnar, open formats (Parquet, and table formats such as Iceberg or Delta) for cost and portability

The pitfall: a swamp with no grain. Everything is technically available and nothing is trustworthy: three tables that each claim to be "orders", none with a stated grain or owner.

Illustrative tools: Snowflake, BigQuery, Redshift (warehouse) · Databricks, Iceberg, Delta Lake (lakehouse) · Parquet.

M Usman · From enterprise data to production AI.

3 · Transform & test: clean, conform, check

The modelling logic written as version-controlled, tested transformations, so a business-ready table is reproducible and its quality is enforced in code.

Practical steps

- Express transformations as reviewed, version-controlled code, not hand-run SQL
- Build the modelled layer as a dependency graph of small, documented models with clear ownership
- Write **data-quality tests beside the transforms**: not-null, unique, accepted values, referential integrity, freshness
- Tier the tests to the layer: strict contracts at the edges, business rules at the gold layer

The pitfall: silent breakage. With no contract test, a renamed column or a half-empty load propagates corrupted data downstream, and the dashboard is wrong before anyone asks why.

Illustrative tools: dbt, SQLMesh (transformation) · Great Expectations, Soda, dbt tests (quality as code).

M Usman · From enterprise data to production AI.

4 · Orchestrate & observe: schedule, retry, alert

The pipeline run as a managed dependency graph that handles order, failure, and lateness on its own, and reports the moment freshness or volume drifts.

Practical steps

- Orchestrate as a **dependency graph**, not a chain of cron jobs: each step runs when its inputs are ready
- Make failure routine: retries with back-off, alerting, and a clear restart point, so 3am is rare and recoverable
- Monitor freshness, volume, and schema, and watch the cost of each pipeline alongside its reliability
- Keep **lineage** end to end, so a broken metric traces to its root node in minutes, not hours

The pitfall: a 3am pager. A chain of unmanaged scripts fails in the dark with no retry and no lineage, and root-causing it is manual archaeology through the dependency graph.

Illustrative tools: Airflow, Dagster, Prefect (orchestration) · Monte Carlo, dbt exposures (observability and lineage).

M Usman · From enterprise data to production AI.

How a data engineer is measured

- **Reliability and freshness against an SLA.** The tables are complete and on time, and a miss is caught before a user sees it, not reported by one.
- **Quality incidents prevented, and time to root-cause.** Bad data is stopped at the contract; when something does break, lineage points straight at the node.
- **Cost per pipeline.** Compute and storage are watched, not just correctness; an efficient pipeline is part of a senior engineer's signature.
- **Latency of fresh data.** The gap between an event and its availability for a decision, held to what the use case needs.

A scripter makes the data move. An engineer makes it move reliably, cheaply, and observably, and proves it.

M Usman · From enterprise data to production AI.

The ladder: from junior to architect



Engineering is rarely an entry point. It is usually reached through software or analytics, and it climbs toward the architect's chair, the senior destination of the data career. UK role averages, Glassdoor 2025/26; bands widen sharply in London and in finance.

M Usman · From enterprise data to production AI.

The roadmap a data engineer climbs

5 · Scale and streaming

Spark for big data; Kafka and streaming; the lakehouse; cost and performance at scale

4 · Orchestration and observability

Airflow, Dagster, or Prefect; CI/CD for data; lineage, monitoring, and SLAs

3 · Transformation and quality

dbt and the modelled layer; tests as code; the medallion pattern on one cloud

2 · Data and SQL

databases, advanced SQL, and data modelling; warehouse and lakehouse foundations

1 · Foundations

Python, version control, and the command line; how data is stored and read

The first two rungs are shared with every data role; the engineer's signature is the climb through pipelines, orchestration, and scale at the top.

M Usman · From enterprise data to production AI.

What good looks like

- **Treat data as a product.** A table is not "done" until it has an owner, a contract, tests, and a stated freshness; availability is not the same as trust.
- **Make quality a property of the code.** Tests live beside the transforms and run on every change, so reliability is versioned, not remembered.
- **Build for the failure, not the demo.** Retries, alerting, lineage, and a restart point are the difference between a pipeline and a liability.
- **Watch the cost.** At scale, an unwatched pipeline is a budget leak; efficiency is a senior engineering skill, not an afterthought.

M Usman · From enterprise data to production AI.

The lens behind this view

My view of this stage comes from eighteen years building and governing the data foundations that pipelines exist to serve. Across multiple ERP estates I led master data as a discipline. Customer, product, vendor, and ledger records were resolved to one authoritative key, with nomenclature and lifecycle standards written into the business specifications. On that foundation I designed the governed BI and reporting layer end to end, the modelled, decision-ready tables a pipeline feeds into. I also teach the database foundations the role is built on.

The tools have changed faster than the discipline. The contract, the modelled layer, the quality check, and the lineage are problems I have worked on for years. Expressing them in dbt, an orchestrator, and a lakehouse instead of an ERP estate is the part I am building now.

The research and the worked examples behind this view are public: mtusman-ai.github.io/M.Usman

M Usman · From enterprise data to production AI.

In one line

**The data engineer builds the floor every other data role stands on.
Land it reliably, model it with a grain, test it in code, and orchestrate it
so it watches itself. The reward is the failure nobody ever sees.**

M Usman · From enterprise data to production AI.

Sources

WEF Future of Jobs (2025): big-data specialists among the fastest-growing roles to 2030 · PwC Global AI Jobs Barometer (2025): 56% global AI wage premium · Role-average UK salaries: Glassdoor UK (2025/26), early-career to lead and architect bands. The engineering practice draws on the data-contracts, medallion, and DataOps literature: dbt; the lakehouse and open-table-format references (Iceberg, Delta); Monte Carlo data-observability. The per-stage learning path follows the standard curriculum progression: foundations, then SQL and databases, then transformation and quality, then orchestration, then scale and streaming. Tools named are illustrative, not endorsements.

M Usman · From enterprise data to production AI. · mtusman-ai.github.io/M.Usman

See the full portfolio



The work behind this deck sits on one page: the Insights series, the projects and the code that runs them, and the publications.

mtusman-ai.github.io/M.Usman

M Usman · From enterprise data to production AI.